

TSI_MLIR_EXPORT: compiled prefill + decode from llama-cli

Principle: offload as much as possible to the compiler so it can optimize. Where work can live on the host or in the graph, it goes in the graph.

Example: **SmolLM2-135M** — 30 layers, hidden 576, vocab 49152, 9 query / 3 KV heads, head_dim 64, FFN 1536, tied embeddings, cache capacity `L = n_ctx = 4096` . Replaces `TSI_WHOLEGRAPH=*` .

What happens when you run the command

```
TSI_MLIR_EXPORT=1 llama-cli -m smolLM2-135m-f32.gguf -p "hello world" -n 4
```

- 1. **llama starts normally** — loads the GGUF, allocates its own KV cache, builds and runs graphs as always. Our hook sits at `graph_compute()` , which sees each full forward graph before the backend splits it. llama's behaviour is unchanged; it stays the reference.
- 2. **We allocate our cache once** — two `tsi_alloc` buffers in device DRAM, 45 MiB each, zeroed: `cache_k` and `cache_v` , shaped `[30, 64, 3, 4096]` = `[layer, head_dim, n_head_kv, cell]` .
- 3. **llama's first graph is a warmup** (BOS+EOS, not your prompt). Skipped.
- 4. **The prefill graph arrives** — 2 tokens, positions 0 and 1. Pre-compute we snapshot the ids and weights; llama computes it and that becomes our reference; post-compute we rebuild an equivalent graph, lower to linalg, emit bytecode, compile, run, compare. Prefill **fills** cells 0-1, all layers.
- 5. **llama samples and calls decode** — 1 token, position 2. Same interception, but we also snapshot llama's KV cache *before* compute, when it holds exactly tokens 0-1: the correct decode input. Decode **reads** cells 0-1 and **appends** cell 2.
- 6. **The compiled logits become the result** — handed back to llama, which samples from them and continues. By default nothing is compared against anything: the point is to run the model through the MLIR path, not to check it.

Prefill fills the cache, decode consumes it. Together they are a generation loop in which the host passes two pointers and reads logits — **no K/V ever crosses the host boundary**.

Comparison is layered, each level costing more than the last:

flag	adds	cost
<code>TSI_MLIR_EXPORT=1</code>	export, compile, run; compiled logits are the output	the compiled forward
<code>TSI_MLIR_VERIFY=1</code>	diff compiled vs llama's own logits	reads llama's result
<code>TSI_MLIR_CPU_REF=1</code>	also compute the reconstruction on CPU, for a 3-way split that separates a reconstruction bug from a compilation bug	a full extra CPU forward pass

The reconstructed graph is always *built* — that is where the MLIR comes from — but only *computed* under `TSI_MLIR_CPU_REF` . Note that llama still runs its own per-op graph today regardless, because the hook fires after compute; `TSI_MLIR_VERIFY` only decides whether we look at the result. Skipping llama's compute entirely, so the compiled path is the only one that runs, is a follow-up.

The two signatures

`L = n_ctx = 4096` , so one binary serves any prompt length; the mask decides which cells are live.

```
// PREFILL – whole prompt at once, N = 2. From-scratch by definition, so slots are 0..N-1.
func.func @forward(
  %ids      : tensor<2xi32>          {txe.name = "input_ids"},
  %cache_k  : memref<30x64x3x4096xf16, 1> {txe.name = "cache_k"},
  %cache_v  : memref<30x64x3x4096xf16, 1> {txe.name = "cache_v"}
) -> (tensor<2x49152xf32> {txe.name = "res_0"})
  attributes {llvm.emit_c_interface} // writes cells 0..N-1, in place

// DECODE – one token. pos/slot/mask come from llama, so cell ordering is irrelevant.
func.func @forward(
  %id      : tensor<1xi32>          {txe.name = "input_ids"},
  %pos     : tensor<1xi32>          {txe.name = "pos"}, // RoPE position
  %slot    : index                  {txe.name = "slot"}, // cell to write
  %mask    : tensor<4096xf32>       {txe.name = "mask"}, // cell validity
  %cache_k : memref<30x64x3x4096xf16, 1> {txe.name = "cache_k"},
  %cache_v : memref<30x64x3x4096xf16, 1> {txe.name = "cache_v"}
) -> (tensor<49152xf32> {txe.name = "res_0"})
  attributes {llvm.emit_c_interface} // writes cell %slot, in place
```

Prefill **4 pointer args**, decode **7** — down from 397 as built today, via three reductions:

	removes
weights are <code>dense_resource</code> constants in the IR; no flag to disable	272 args
cache is a DRAM memref written in place, so argument-only, never a result	60 results
one buffer per kind, not per layer (layers unroll, so the layer index is constant)	58 args

Cache rank is 4 because `GGML_MAX_DIMS` is 4, so a rank-5 `[layer, K/V, ...]` buffer could not be a ggml tensor. Weights ship as bytecode, where blobs are raw binary (269 MB f16) not hex (2x). Cache is f16 to match llama bit for bit, with **f32 accumulation in every reduction** — an f16 sum over 2048 elements loses most of its significance.

Reconstructed, not translated. llama's graph uses `FLASH_ATTN_EXT` (one fused op per layer, nothing equivalent in our lowering) and mutates its cache via `SET_ROWS`. So we rebuild the math with unfused attention. Measured on SmoLLM2 prefill: reconstruction vs llama `1.09e-07`, compiled vs reconstruction `2.25e-12`, compiled vs llama `1.10e-07`. So `TSI_MLIR_VERIFY` should expect `~1.1e-07`, essentially all of it unfused-vs-flash — compilation itself is near-exact. That is also why `TSI_MLIR_CPU_REF` is a separate level: the 3-way split rarely locates anything the 2-way diff did not.

Work required

Outside the exporter: `compile_graph_fpga.py:89` uses `read_text()` and must read bytes to carry bytecode; `build_layer` must expose per-layer K/V so prefill can fill the cache (`build_decode_layer` already does); both `host.so` define `@forward` at different arities, so load `RTL_D_LOCAL` with separate handles; delete `TSI_WG_BAKE_WEIGHTS` and `partitionWeights()`.

Implementation rule: **every memref must carry memory space 1** (DRAM). That single omission accounts for every failure hit while validating the shapes above, including one inside a hand-written `bufferization.to_buffer`.

Three things are unverified and should be settled first: that a DRAM buffer's contents survive *between* calls (compiling proves the type, not the semantics, and the whole design rests on it); constant-pool scaling (the largest constant compiled so far is a few KiB, against a model $\sim 10^5$ times larger); and the op bodies themselves.

Order: those three, then f16/bf16 through the dialect and emitters (largest piece, de-risked by existing compiler-side bf16 tests), then `build_layer` K/V, then cache append and constants-as-bytecode, then the driver and SmoLLM2 verification. An f32-only path through the middle steps reaches a working result sooner, at the cost of reworking the cache element type later.

llama-server compatibility

`llama-cli` is single-user and single-sequence. `llama-server` is the concurrent one — `server_slot`, `n_slots = n_parallel`, `n_ctx_slot = n_ctx / n_parallel`. **This design does not work there**, for two reasons:

1. **Shape bucketing instead of two fixed graphs.** A server ubatch mixes tokens from several requests, so `n_tokens` varies per call and each value is a distinct graph. Compile a fixed set (1, 8, 64, 512), pad up to the nearest, and key compiled artifacts by shape rather than by phase.
2. **Per-token sequence routing.** One ubatch carries tokens for different slots at different positions, so `slot / pos / mask` must become per-token vectors rather than the scalars decode uses here, and the cache append becomes a scatter rather than one subview.

Carries over: weights-as-constants, the DRAM in-place cache, and the mask-as-input decision, which already made the graph agnostic to cache layout — the prerequisite for multi-sequence. Breaks: the two-latch capture is llama-cli-shaped, `L = n_ctx` conflicts with the per-slot context split, and the clean prefill/decode dichotomy does not exist there at all.

Item 2 is a different reconstruction, not an extension of this one: this design builds a graph for a single token stream, whereas a server graph is inherently ragged. Treat llama-cli as the deliberate first target — it isolates the compiler work from the serving work.